

Dokumentacja Projektu 5 (SIGK)

Autorzy:

- Peng-Yin Chen
 - Paweł Niewierowski
-

1. Cel projektu

Celem projektu było stworzenie modelu opartego na sieciach dyfuzyjnych, który generuje animacje 3D szkieletu człowieka (tzw. stickmana). Projekt zrealizowano w języku Python z wykorzystaniem biblioteki PyTorch.

Główne założenia:

- **Wejście:** Komenda tekstowa od użytkownika określająca typ ruchu: chód (*walk*) lub skok (*jump*).
- **Dane treningowe:** Wykorzystano zbiór danych CMU MoCap (pliki *.bvh*).
- **Format wyjściowy:** Tensor o wymiarach [48, 15, 3], co odpowiada 48 klatkom animacji dla 15 punktów kluczowych szkieletu w trójwymiarowej przestrzeni.
- **Wizualizacja:** Wyświetlenie wygenerowanych współrzędnych w formie połączanego szkieletu 3D.
- **Ewaluacja:** Ocena jakości i kreatywności modelu za pomocą trzech metryk: Fréchet Motion Distance (FMD), Mean Per Joint Position Error (MPJPE) oraz wariancji (Var).

2. Metodologia i zbiór danych

Proces uczenia oparto na zasobach **CMU MoCap**, stanowiących bazę nagrań kinetyki człowieka utrwaloną w rozszerzeniu *.bvh*. Zbiór ten umożliwił wyodrębnienie konkretnych serii ruchowych dla analizowanych czynności: chodu (*walk*) oraz skoku (*jump*).

2.1. Przetwarzanie danych

Dane surowe z formatu *.bvh* wymagały zaawansowanego preprocessingu. W ramach prac przygotowano **dwie wersje zbioru danych**: oryginalną (poddaną jedynie podstawowej ekstrakcji) oraz docelową, która przeszła pełen proces filtrowania i normalizacji w celu poprawy wyników sieci dyfuzyjnej.

Główne etapy przetwarzania obejmowały:

- **Ekstrakcja i downsampling:** Z plików *.bvh* za pomocą kinematyki prostej (*forward kinematics*) wyodrębniono współrzędne 3D dla 15 punktów kluczowych szkieletu. Ruch nagrany w 120 FPS zredukowano do docelowych 24 FPS.
- **Podział na sekwencje i filtr ruchu (wersja docelowa):** Dane podzielono na 48-klatkowe okna. W docelowej wersji zbioru zastosowano algorytm filtrujący (bazujący na progu wariancji), który automatycznie odrzucał okna ze statycznymi pozami (np. aktor stojący w bezruchu przed wykonaniem akcji). Dzięki temu model uczył się wyłącznie właściwej dynamiki ruchu.

- **Normalizacja przestrzenna (wersja docelowa):** Każda 48-klatkowa sekwencja została wyśrodkowana względem pozycji miednicy (*Pelvis*) w zerowej klatce. Osie zostały zorientowane pionowo (Z-up), a wartości współrzędnych odpowiednio przeskalowane.
- **Standaryzacja dla dyfuzji (wersja docelowa):** Ponieważ model dyfuzyjny operuje na standardowym szumie Gaussa $N(0, 1)$, docelowy zbiór danych został ustandaryzowany do średniej równej 0 i odchylenia standardowego równego 1 (proces ten jest automatycznie odwracany podczas generowania wyników końcowych).

2.2. Metryki oceny

Do obiektywnej weryfikacji jakości wygenerowanego ruchu i kreatywności modelu zastosowano trzy miary:

- **Fréchet Motion Distance (FMD):** Mierzy podobieństwo dystrybucji ruchu rzeczywistego do wygenerowanego. Ze względu na wysoką wymiarowość danych (2160 wartości na sekwencję), przed obliczeniami zastosowano redukcję wymiarowości algorytmem **PCA** do 32 komponentów. Zapewniło to stabilność matematyczną i precyzyjny wynik metryki.
- **Mean Per Joint Position Error (MPJPE):** Średni euklidesowy błąd pozycjonowania stawów. Aby uniknąć karania modelu za globalne przesunięcia (np. rozpoczęcie chodu w innym punkcie przestrzeni), przed kalkulacją błędu wygenerowany i rzeczywisty szkielet są wyrównywane względem miednicy w pierwszej klatce.
- **Wariancja (Var):** Mierzy różnorodność generowanych sekwencji. Wyższa wariancja świadczy o tym, że model nie kopiuje sztywno danych treningowych, lecz generuje kreatywne i unikalne warianty ruchu (np. różne style chodzenia) dla tego samego wejścia tekstowego.

3. Architektura modelu i proces treningu

Do generowania ruchu wykorzystano model z rodziny **Denoising Diffusion Probabilistic Models (DDPM)**. Rozwiązanie składa się z dwóch głównych komponentów: procesu dyfuzji dodającego szum oraz sieci neuronowej uczącej się go usuwać.

3.1. Architektura sieci odszumiającej (Motion Denoiser)

Architektura modelu bazuje na jednowymiarowej, konwolucyjnej sieci neuronowej (1D Conv), specjalizującej się w analizie danych sekwencyjnych.

- **Wejście i wyjście:** Sieć przyjmuje i zwraca tensor o wymiarach [Batch, 45, 48], gdzie 45 to spłaszczone współrzędne 3D dla 15 punktów kluczowych szkieletu, a 48 to liczba klatek.
- **Warunkowanie (Conditioning):** Generowanie ruchu jest warunkowane na dwa sposoby:
 1. **Krok czasowy (Timestep):** Kodowany za pomocą osadzeń sinusoidalnych (*sinusoidal time embeddings*), co pozwala sieci zlokalizować, na jakim etapie odszumiania się znajduje.

2. **Etykieta klasy:** Tekstowy prompt użytkownika (*walk* lub *jump*) jest zamieniany na indeks i wprowadzany do sieci przez warstwę `nn.Embedding`.
- **Budowa:** Przetwarzanie sygnału odbywa się w warstwach rezydualnych (`ResidualBlock1d`), wykorzystujących konwolucję, normalizację grupową (`GroupNorm`) oraz nieliniową funkcję aktywacji SiLU. Warunkowanie (czas + etykieta) jest dodawane jako wektor do każdej warstwy ukrytej.

3.2. Proces dyfuzji (DDPM)

Proces niszczenia i odtwarzania danych zdefiniowano za pomocą liniowego harmonogramu wariancji:

- Liczba kroków dyfuzji (T) wynosi **1000**.
- Stopień dodawania szumu Gaussa $N(0, 1)$ rośnie liniowo od $\beta_{start} = 10^{-4}$ do $\beta_{start} = 2 \times 10^{-4}$.
- Podczas generowania (inferencji) model iteracyjnie, krok po kroku, usuwa wyuczony szum z początkowego tensora całkowicie losowych wartości, tworząc płynną animację.

3.3. Proces uczenia

Trening modelu polegał na minimalizacji błędu średniokwadratowego (MSE) pomiędzy sztucznie dodanym szumem Gaussa a szumem przewidzianym przez sieć dla danego kroku czasowego.

- **Optymalizator i Scheduler:** Zastosowano optymalizator **AdamW** ze współczynnikiem uczenia na poziomie 2×10^{-4} oraz regularyzacją `weight_decay`. Stopniowe wygaszanie tempa uczenia realizowano za pomocą strategii *Cosine Annealing*.
- **Zarządzanie partiami (Batching):** Rozmiar batcha wynosił 64. Aby zapobiec faworyzowaniu jednej z klas (w przypadku nierównej liczby próbek dla chodu i skoku), zastosowano mechanizm `WeightedRandomSampler`, który dbał o balans klas podczas losowania próbek.
- **Czas treningu:** Z uwagi na wysoką złożoność zadania i powolną zbieżność modeli dyfuzyjnych w generacji dynamiki 3D, liczba epok została zwiększona z **1000 do 3000 epoka**. Pozwoliło to na dogłębne wyuczenie gęstego manifold'u ruchu i wyeliminowało problem tzw. "eksplozji współrzędnych" (rozpadu szkieletu) podczas inferencji.

4. Wyniki eksperymentów

4.1. Bez filtrowania wariancji ruchu ani normalizacji

Typ	FMD	MPJPE	Var
Walk (epoch=1000)	2.265110	0.035776	0.001482
Jump (epoch=1000)	4.012441	0.041777	0.002204

4.2. Z filtrowaniem wariancji ruchu i normalizacją

Typ	FMD	MPJPE	Var
Walk (epoch=1000)	5.447889	0.085659	0.005026
Jump (epoch=1000)	7.007618	0.092991	0.005103
Walk (epoch=2000)	4.525446	0.078386	0.004041
Jump (epoch=2000)	5.564455	0.082500	0.003903
Walk (epoch=3000)	4.217092	0.074303	0.003132
Jump (epoch=3000)	5.241120	0.082640	0.003727

5. Wizualizacja

5.1 Bez filtrowania wariancji ruchu ani normalizacji

Jump:

https://github.com/nitrazek/AI-Computer-Graphics/blob/main/project_5/results/generated/jump_02.gif

Walk:

https://github.com/nitrazek/AI-Computer-Graphics/blob/main/project_5/results/generated/walk_02.gif

5.2 Z filtrowaniem wariancji ruchu i normalizacją

Jump:

https://github.com/nitrazek/AI-Computer-Graphics/blob/main/project_5/results/generated_test/jump_01.gif

Walk:

https://github.com/nitrazek/AI-Computer-Graphics/blob/main/project_5/results/generated_test/walk_02.gif

6. Analiza wizualna i wnioski

6.1. Wyniki metryk

Zestawienie wyników modelu przed optymalizacją potoku danych (Tabela 4.1) oraz po jej zastosowaniu (Tabela 4.2) doskonale obrazuje jedną z najczęstszych pułapek w trenowaniu generatywnych sieci neuronowych.

- **Pułapka statycznego modelu (Przed optymalizacją):** Wartości wariancji (Var ~0.001) oraz błędu przestrzennego (MPJPE ~0.035) w Tabeli 4.1 wydawały się początkowo doskonałe. W rzeczywistości wynikały one z faktu, że zbiór danych był

zdominowany przez statyczne klatki (aktor stojący w bezruchu). Model zoptymalizował funkcję straty, ucząc się generować jedną, "zamrożoną" pozę.

- **Poprawa zbieżności w czasie (Po optymalizacji):** Po wdrożeniu filtra ruchu oraz normalizacji danych (Tabela 4.2), metryki stały się wiarygodne. Analiza epok (1000, 2000, 3000) ukazuje stabilny proces uczenia:
 - **FMD (Fréchet Motion Distance):** Dla ruchu chodu (*walk*) wartość spadła z 5.44 (epoka 1000) do 4.21 (epoka 3000), co dowodzi, że generowany ruch coraz lepiej naśladował rzeczywistą dystrybucję danych MoCap.
 - **MPJPE:** Błąd pozycjonowania stawów również systematycznie malał, osiągając świetny wynik 0.074 dla chodu i 0.082 dla skoku w 3000. epoce.

6.2. Analiza wizualna i generalne wnioski

Wizualna inspekcja wygenerowanych animacji 3D potwierdza wyniki liczbowe i pozwala sformułować następujące wnioski końcowe:

- **Kluczowa rola preprocessingu:** Modele dyfuzyjne są niezwykle wrażliwe na jakość danych. Usunięcie "martwych okien" (statycznych klatek) za pomocą filtra wariancji było niezbędne, aby zmusić model do nauki rzeczywistej dynamiki chodu i skoku, a nie tylko odtwarzania sztywnej postawy.
- **Konieczność normalizacji:** Standardowy proces dyfuzji (DDPM) opiera się na szumie Gaussa o średniej 0 i odchyleniu 1. Próba uczenia na surowych, nieustandaryzowanych współrzędnych przestrzennych prowadziła do "eksplozji" modelu podczas generowania (olbrzymie wartości błędów). Wdrożenie pełnej normalizacji zbioru całkowicie wyeliminowało ten problem.
- **Czas treningu (Epoki):** Nauka złożonych, czasowo-przestrzennych zależności ludzkiej motoryki z czystego szumu wymaga znacznego czasu. Wyniki pokazują, że standardowe 200-1000 epok to za mało dla tego zadania. Znacząca poprawa płynności i realizmu animacji (widoczna w spadku FMD) nastąpiła dopiero po wydłużeniu treningu do 3000 epok.

Podsumowując, zaimplementowane rozwiązanie z powodzeniem realizuje cel projektu, generując płynne i zróżnicowane animacje chodu i skoku warunkowane tekstem, co potwierdzają zarówno niskie wartości FMD i MPJPE, jak i poprawna wizualizacja 3D.